1120221303 俞乐楠 实验四 实验报告

一、程序中使用的数据结构及符号说明

符号	类型	说明
`Directory`	类	目录节点，包含目录名、父目录指针、子目录列表(`subdirs`)与文件列表(`files`)
`File`	类	文件对象，包含文件名(`name`)和内容(`content`)
`root`	`Directory*`	根目录指针
`current`	`Directory*`	当前工作目录指针
`openFile`	`File*`	当前打开的文件指针
`lastFilename`	`string`	最近保存或加载的文件名
`D` , `END` , `F`	文本标记	序列化格式中表示目录开始/结束和文件条目的标记

二、模块设计及功能接口说明

2.1 命令解析模块（CommandParser）

- **功能**：从用户输入中解析命令和参数，分发到对应处理函数。
- **接口**：

  ```
  bool parseAndExecute(const string &input);
  ```

  - **参数**： `input` 为整行命令字符串。
  - **返回**：执行结果是否继续循环。

2.2 目录管理模块（DirectoryManager）

- **功能**：实现目录的增删查改，包括 `mkdir` 、 `rmdir` 、 `ls` 、 `cd` 。
- **接口**：

```
void makeDirectory(Directory *current, const string &name);
void removeDirectory(Directory *current, const string &name);
vector<string> listDirectory(const Directory *current);
Directory* changeDirectory(Directory *current, const string &name);
```

2.3 文件管理模块（FileManager）

- **功能**：实现文件的创建、打开、写入、读取、关闭、删除。
- **接口**：

```
File* createFile(Directory *current, const string &filename);
File* openFile(Directory *current, const string &filename);
void writeFile(File *f, const string &text);
string readFile(const File *f);
void closeFile(File *&openFile);
void deleteFile(Directory *current, const string &filename);
```

2.4 持久化模块（PersistenceManager）

- **功能**：将内存中的目录树和文件内容序列化保存到磁盘；从磁盘反序列化加载到内存。
- **接口**：

```
void saveDirectory(ofstream &ofs, Directory *dir);
void parseDirectory(ifstream &ifs, Directory *dir);
Directory* loadFileSystem(const string &filename);
```

三、命令分发与执行流程说明

> 本实验中所谓“调度算法”即命令解析和模块分发逻辑。

1. 用户输入命令后，由**命令解析模块**识别关键字。
2. 将命令和参数传递给相应模块处理：
    - 目录操作: **DirectoryManager**
    - 文件操作: **FileManager**
    - 保存/加载: **PersistenceManager**
3. 处理完成后返回状态，决定命令循环是否继续。

流程示意图：

```
┌──────────────┐      ┌──────────────────┐
│ CommandParser │──────>│ DirectoryManager  │
└──────────────┘      └──────────────────┘
        │                      │
        │                      ▼
        │              ┌──────────────┐
        └──────────────>│ FileManager  │
                       └──────────────┘
        │                      │
        └──────────────────────>▼
                       ┌──────────────────┐
                       │ PersistenceManager │
                       └──────────────────┘
```

四、源程序及主要注释

代码已在 `main.cpp` 文件中完整提供，以下为核心片段及注释示例：

```
// 递归保存目录
void saveDirectory(ofstream &ofs, Directory* dir) {
    ofs << "D " << dir->name << "\n";
    for (File* f : dir->files) {
        ofs << "F " << f->name << " " << f->content << "\n";
    }
    for (Directory* sub : dir->subdirs) saveDirectory(ofs, sub);
    ofs << "END\n";
}
```

五、测试方法与结果分析

5.1 文件夹处理

`mkdir, rmdir, ls, cd`

创建目录、删除目录、展示目录、打开目录

```
root/docs>mkdir docs
mkdir media
目录 'docs' 创建成功。
root/docs>

目录 'media' 创建成功。
root/docs>ls

docs/
media/
root/docs>rmdir docs

目录 'docs' 已删除。
root/docs>ls

media/
root/docs>cd docs

子目录 'docs' 不存在。
root/docs>cd media

root/docs/media>
```

5.2 文件处理

`create, open, close, read, write, delete`
文件创建/删除、开/关、读/写

```
用法: cd <目录名>
root/docs/media>create report.txt

当前目录下已存在同名文件或目录。
root/docs/media>create report1.txt

文件 'report1.txt' 创建成功。
root/docs/media>open report1.txt

文件 'report1.txt' 已打开。
root/docs/media>write 本实验实现了一个虚拟文件系统。支持文件的读写、目录嵌套、保存与加载。

写入完成。
root/docs/media>read report1.txt

本实验实现了一个虚拟文件系统。支持文件的读写、目录嵌套、保存与加载。
root/docs/media>close

文件 'report1.txt' 已关闭。
```

5.3 文件系统处理

`new, sfs <file>, exit`

文件系统创建、打开、退出

创建文件系统，打开

```
root>new

已创建新文件系统。
root>mkdir docs
mkdir media
目录 'docs' 创建成功。
root>

目录 'media' 创建成功。
root>

root>ls

docs/
media/
root>exit myfs.dat

文件系统已保存到: myfs.dat
退出虚拟文件系统。

进程已结束，退出代码为 0
```

再次打开后

```
D:\Tasks\操作系统\Experiment4\cmake-build-debug\Experiment4.exe
虚拟文件系统模拟（命令示例：new, sfs <file>, exit, mkdir, rmdir, ls, cd, create, open, close, read, write, delete）
root>sfs myfs.dat

从文件加载文件系统: myfs.dat
root>ls

docs/
media/
root>
```

六、本次实验经验及体会

1. **模块化编程**：分层设计提高可读性和可维护性。
2. **树形结构理解**：深刻体会文件系统目录树的生成与遍历。
3. **持久化思路**：掌握了简单的序列化/反序列化实现。
4. **调试与测试**：通过系统测试用例完善了异常处理逻辑。

七、具体实现代码

```
cmake_minimum_required(VERSION 3.29)
project(Experiment4)

set(CMAKE_CXX_STANDARD 20)

add_executable(Experiment4 main.cpp)
```

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <sstream>
#include <fstream>

using namespace std;

// 表示文件的类，保存文件名和内容
class File {
public:
    string name;
    string content;
    File(string name) : name(name), content("") {}
};

// 表示目录的类，包含子目录和文件列表
class Directory {
public:
    string name;                        // 目录名
    vector<Directory*> subdirs;         // 子目录列表
    vector<File*> files;                // 文件列表
    Directory* parent;                  // 父目录指针（根目录的 parent 为 nullptr）

    Directory(string name, Directory* parent = nullptr)
        : name(name), parent(parent) {}

    // 析构函数递归释放子目录和文件的内存
    ~Directory() {
        for (File* f : files) {
            delete f;
        }
        for (Directory* d : subdirs) {
            delete d;
        }
    }
};

// 递归地将目录结构保存到文件流中
void saveDirectory(ofstream &ofs, Directory* dir) {
    ofs << "D " << dir->name << "\n";  // 写入当前目录标识
    // 保存当前目录下的所有文件
    for (File* f : dir->files) {
        ofs << "F " << f->name;
        if (!f->content.empty()) {
            ofs << " " << f->content;
```

```
        }
        ofs << "\n";
    }
    // 递归保存所有子目录
    for (Directory* sub : dir->subdirs) {
        saveDirectory(ofs, sub);
    }
    ofs << "END\n";  // 标记当前目录结束
}

// 递归地从文件流解析目录结构到内存
void parseDirectory(ifstream &ifs, Directory* dir) {
    string line;
    while (getline(ifs, line)) {
        if (line == "END") {
            // 读到 END 表示当前目录结束
            return;
        }
        if (line.rfind("F ", 0) == 0) {
            // 文件条目，格式：F 文件名 文件内容
            string rem = line.substr(2);
            size_t pos = rem.find(' ');
            string fname, fcontent;
            if (pos != string::npos) {
                fname = rem.substr(0, pos);
                fcontent = rem.substr(pos + 1);
            } else {
                fname = rem;
                fcontent = "";
            }
            File* f = new File(fname);
            f->content = fcontent;
            dir->files.push_back(f);
        }
        else if (line.rfind("D ", 0) == 0) {
            // 子目录开始条目，格式：D 目录名
            string subName = line.substr(2);
            Directory* sub = new Directory(subName, dir);
            dir->subdirs.push_back(sub);
            parseDirectory(ifs, sub);  // 递归解析子目录内容
        }
        // 其他行忽略
    }
}

// 从文件加载文件系统，返回根目录指针（外部需管理该内存）
```

```
Directory* loadFileSystem(const string &filename) {
    ifstream ifs(filename);
    if (!ifs) {
        cout << "无法打开文件来加载文件系统: " << filename << "\n";
        return nullptr;
    }
    string line;
    if (!getline(ifs, line)) {
        cout << "文件为空或格式不正确。\n";
        return nullptr;
    }
    if (line.rfind("D ", 0) != 0) {
        cout << "无效的文件系统格式。\n";
        return nullptr;
    }
    string rootName = line.substr(2);
    Directory* root = new Directory(rootName, nullptr);
    parseDirectory(ifs, root);
    return root;
}

int main() {
    Directory* root = nullptr;       // 根目录指针
    Directory* current = nullptr;    // 当前工作目录指针
    File* openFile = nullptr;        // 当前打开的文件指针
    string lastFilename = "";        // 最近保存/加载的文件名

    cout << "虚拟文件系统模拟（命令示例: new, sfs <file>, exit, mkdir, rmdir, ls, cd, create, op

    // 默认创建一个新文件系统
    root = new Directory("root", nullptr);
    current = root;

    while (true) {
        // 打印提示符（显示当前路径）
        vector<string> path;
        Directory* tmp = current;
        while (tmp) {
            path.push_back(tmp->name);
            tmp = tmp->parent;
        }
        string fullpath;
        for (auto it = path.rbegin(); it != path.rend(); ++it) {
            fullpath += *it;
            if (it + 1 != path.rend()) fullpath += "/";
        }
```

```
cout << fullpath << "> ";

string line;
if (!getline(cin, line)) {
    break;  // 读入失败（EOF）时退出
}
if (line.empty()) {
    continue;
}
// 解析命令
istringstream iss(line);
string cmd;
iss >> cmd;

if (cmd == "new") {
    // 重置文件系统
    if (root) delete root;
    root = new Directory("root", nullptr);
    current = root;
    openFile = nullptr;
    lastFilename = "";
    cout << "已创建新文件系统。\n";
}
else if (cmd == "sfs") {
    // 从指定文件加载文件系统
    string fname;
    iss >> fname;
    if (fname.empty()) {
        fname = "filesystem.dat";  // 默认文件名
    }
    Directory* loaded = loadFileSystem(fname);
    if (loaded) {
        if (root) delete root;
        root = loaded;
        current = root;
        openFile = nullptr;
        lastFilename = fname;
        cout << "从文件加载文件系统: " << fname << "\n";
    } else {
        cout << "加载失败，保持当前文件系统不变。\n";
    }
}
else if (cmd == "exit") {
    // 保存文件系统并退出
    string fname;
    iss >> fname;
```

```
        if (fname.empty()) {
            if (!lastFilename.empty()) {
                fname = lastFilename;
            } else {
                fname = "filesystem.dat"; // 默认文件名
            }
        }
        ofstream ofs(fname);
        if (!ofs) {
            cout << "无法打开文件保存: " << fname << "\n";
        } else {
            saveDirectory(ofs, root);
            cout << "文件系统已保存到: " << fname << "\n";
        }
        if (root) delete root;
        break;
    }
    else if (cmd == "mkdir") {
        // 创建子目录
        string dirname;
        iss >> dirname;
        if (dirname.empty()) {
            cout << "用法: mkdir <目录名>\n";
            continue;
        }
        // 检查重名
        bool exists = false;
        for (auto d : current->subdirs) {
            if (d->name == dirname) { exists = true; break; }
        }
        for (auto f : current->files) {
            if (f->name == dirname) { exists = true; break; }
        }
        if (exists) {
            cout << "当前目录下已存在同名文件或目录。\n";
        } else {
            Directory* nd = new Directory(dirname, current);
            current->subdirs.push_back(nd);
            cout << "目录 '" << dirname << "' 创建成功。\n";
        }
    }
    else if (cmd == "rmdir") {
        // 删除子目录（仅当空时）
        string dirname;
        iss >> dirname;
        if (dirname.empty()) {
```

```
            cout << "用法: rmdir <目录名>\n";
            continue;
        }
        int idx = -1;
        for (int i = 0; i < (int)current->subdirs.size(); ++i) {
            if (current->subdirs[i]->name == dirname) {
                idx = i;
                break;
            }
        }
        if (idx < 0) {
            cout << "未找到子目录 '" << dirname << "'。\n";
        } else {
            Directory* target = current->subdirs[idx];
            if (!target->subdirs.empty() || !target->files.empty()) {
                cout << "目录 '" << dirname << "' 非空，无法删除。\n";
            } else {
                current->subdirs.erase(current->subdirs.begin() + idx);
                delete target;
                cout << "目录 '" << dirname << "' 已删除。\n";
            }
        }
    }
    else if (cmd == "ls") {
        // 列出当前目录下的所有子目录和文件
        for (auto d : current->subdirs) {
            cout << d->name << "/\n";
        }
        for (auto f : current->files) {
            cout << f->name << "\n";
        }
    }
    else if (cmd == "cd") {
        // 切换目录
        string dirname;
        iss >> dirname;
        if (dirname.empty()) {
            cout << "用法: cd <目录名>\n";
            continue;
        }
        if (dirname == "..") {
            if (current->parent) {
                current = current->parent;
            } else {
                cout << "已在根目录。\n";
            }
```

```
        } else {
            bool found = false;
            for (auto d : current->subdirs) {
                if (d->name == dirname) {
                    current = d;
                    found = true;
                    break;
                }
            }
            if (!found) {
                cout << "子目录 '" << dirname << "' 不存在。\n";
            }
        }
    }
    else if (cmd == "create") {
        // 创建文件
        string filename;
        iss >> filename;
        if (filename.empty()) {
            cout << "用法: create <文件名>\n";
            continue;
        }
        bool exists = false;
        for (auto f : current->files) {
            if (f->name == filename) { exists = true; break; }
        }
        for (auto d : current->subdirs) {
            if (d->name == filename) { exists = true; break; }
        }
        if (exists) {
            cout << "当前目录下已存在同名文件或目录。\n";
        } else {
            File* f = new File(filename);
            current->files.push_back(f);
            cout << "文件 '" << filename << "' 创建成功。\n";
        }
    }
    else if (cmd == "open") {
        // 打开文件（标记为当前操作文件）
        string filename;
        iss >> filename;
        if (filename.empty()) {
            cout << "用法: open <文件名>\n";
            continue;
        }
        if (openFile != nullptr) {
```

```
            cout << "已打开一个文件，请先 close。\n";
            continue;
        }
        File* ftarget = nullptr;
        for (auto f : current->files) {
            if (f->name == filename) {
                ftarget = f;
                break;
            }
        }
        if (!ftarget) {
            cout << "文件 '" << filename << "' 不存在。\n";
        } else {
            openFile = ftarget;
            cout << "文件 '" << filename << "' 已打开。\n";
        }
    }
    else if (cmd == "close") {
        // 关闭当前打开文件
        if (openFile == nullptr) {
            cout << "当前没有打开的文件。\n";
        } else {
            cout << "文件 '" << openFile->name << "' 已关闭。\n";
            openFile = nullptr;
        }
    }
    else if (cmd == "read") {
        // 读取当前打开文件内容
        if (openFile == nullptr) {
            cout << "当前没有打开的文件。\n";
        } else {
            if (openFile->content.empty()) {
                cout << "文件为空。\n";
            } else {
                cout << openFile->content << "\n";
            }
        }
    }
    else if (cmd == "write") {
        // 写入文本到当前打开文件
        size_t pos = line.find(' ');
        string text = (pos != string::npos ? line.substr(pos + 1) : "");
        if (openFile == nullptr) {
            cout << "当前没有打开的文件。\n";
        } else {
            openFile->content += text;
```

```
                cout << "写入完成。\n";
            }
        }
        else if (cmd == "delete") {
            // 删除文件（当前目录下）
            string filename;
            iss >> filename;
            if (filename.empty()) {
                cout << "用法: delete <文件名>\n";
                continue;
            }
            int idx = -1;
            for (int i = 0; i < (int)current->files.size(); ++i) {
                if (current->files[i]->name == filename) {
                    idx = i;
                    break;
                }
            }
            if (idx < 0) {
                cout << "文件 '" << filename << "' 不存在。\n";
            } else {
                if (openFile == current->files[idx]) {
                    cout << "无法删除当前已打开的文件。\n";
                } else {
                    File* target = current->files[idx];
                    current->files.erase(current->files.begin() + idx);
                    delete target;
                    cout << "文件 '" << filename << "' 已删除。\n";
                }
            }
        }
        else {
            cout << "未知命令: " << cmd << "\n";
        }
    }

    cout << "退出虚拟文件系统。\n";
    return 0;
}
```